

# Proyecto Integrador: Dashboard Interactivo

## Descripción

Este es el **proyecto final de la Etapa 2 (Fundamentos de React)** del bootcamp. Construirás un **Dashboard Interactivo** que integre todos los conceptos aprendidos durante las semanas 2-6.

El dashboard debe demostrar dominio de:

- Custom hooks tipados
- Composición de componentes
- Context API para estado global
- TypeScript estricto
- Patrones de React modernos

## Tu Dominio Asignado

El instructor te asignará un dominio único. Algunos ejemplos:

| Dominio                                                                                         | Widgets de ejemplo                                          |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
|  Biblioteca    | Libros prestados, lectores activos, devoluciones pendientes |
|  Farmacia      | Medicamentos bajo stock, ventas del día, recetas pendientes |
|  Gimnasio     | Miembros activos, clases del día, equipos en mantenimiento  |
|  Escuela     | Asistencia, calificaciones, tareas pendientes               |
|  Restaurante | Pedidos activos, mesas ocupadas, inventario bajo            |
|  Hospital    | Pacientes en espera, camas disponibles, citas del día       |

## Objetivos del Proyecto

1. **Implementar 5+ custom hooks** aplicados a tu dominio
2. **Crear 3+ widgets** que muestren información relevante
3. **Usar Context API** para preferencias del usuario
4. **Aplicar TypeScript estricto** en todo el proyecto
5. **Organizar código** siguiendo arquitectura escalable

## Estructura del Proyecto

```
3-proyecto/
├── README.md                # Este archivo
├── starter/
│   ├── src/
│   │   ├── hooks/          # Custom hooks
│   │   │   ├── useLocalStorage.ts
│   │   │   ├── useFetch.ts
│   │   │   ├── useToggle.ts
│   │   │   └── index.ts     # Barrel export
│   │   ├── context/
│   │   │   ├── ThemeContext.tsx
│   │   │   └── index.ts
│   │   ├── components/
│   │   └── widgets/        # Widgets del dashboard
```

```

├── README.md      # Instrucciones
├── layout/
│   ├── Header.tsx
│   └── Sidebar.tsx
├── ui/
│   ├── Card.tsx
│   ├── Button.tsx
│   └── Spinner.tsx
├── types/
│   └── index.ts
├── App.tsx
├── main.tsx
├── index.html
├── package.json
├── tsconfig.json
├── vite.config.ts
└── solution/      # Solución de referencia (genérica)

```

## ✓ Requisitos Funcionales

### 1. Custom Hooks (obligatorios)

Debes implementar **al menos 5** custom hooks:

| Hook               | Descripción                  | Uso en dashboard          |
|--------------------|------------------------------|---------------------------|
| useLocalStorage<T> | Persistencia de preferencias | Tema, sidebar collapsed   |
| useFetch<T>        | Llamadas a API               | Cargar datos del dominio  |
| useToggle          | Boolean toggle               | Sidebar, modals           |
| useDebounce        | Optimización                 | Búsqueda en tiempo real   |
| useFilter<T>       | Filtrar arrays               | Filtrar items del dominio |

### 2. Widgets (mínimo 3)

Cada widget debe:

- Mostrar datos relevantes del dominio
- Usar al menos un custom hook
- Tener estados de loading/error
- Ser visualmente distintivo

**Ejemplos por dominio:**

- **Biblioteca:** Libros más prestados, Préstamos vencidos, Nuevas adquisiciones
- **Farmacia:** Medicamentos bajo stock, Ventas por categoría, Alertas de caducidad
- **Gimnasio:** Check-ins del día, Clases populares, Membresías por vencer

### 3. Context API

Implementar `ThemeContext` con:

- Tema claro/oscuro
- Colores primarios configurables
- Persistencia en localStorage

4. Layout Responsive

- Header con título y controles
- Sidebar colapsable
- Grid de widgets adaptable

 Instrucciones Paso a Paso

Paso 1: Setup (10 min)

```
cd 3-proyecto/starter
pnpm install
pnpm dev
```

Paso 2: Implementar Hooks (30 min)

1. Completa los hooks en `src/hooks/`
2. Verifica que los tipos sean correctos
3. Exporta desde `index.ts`

Paso 3: Configurar Context (20 min)

1. Implementa `ThemeContext` con provider
2. Agrega tipos para el tema
3. Conecta con `useLocalStorage`

Paso 4: Crear Widgets (40 min)

1. Define tipos para tu dominio en `src/types/`
2. Crea 3 widgets en `src/components/widgets/`
3. Usa hooks para data fetching
4. Aplica estilos del tema

Paso 5: Integrar Dashboard (20 min)

1. Configura el layout en `App.tsx`
2. Renderiza los widgets en grid
3. Conecta sidebar y header
4. Prueba responsive design

 Rúbrica de Evaluación

| Criterio      | Peso | Descripción                         |
|---------------|------|-------------------------------------|
| Custom Hooks  | 25%  | 5+ hooks funcionales y tipados      |
| Widgets       | 25%  | 3+ widgets con loading/error states |
| Context API   | 15%  | Tema funcional con persistencia     |
| TypeScript    | 15%  | Tipos estrictos, sin any            |
| UI/UX         | 10%  | Diseño coherente y responsive       |
| Código Limpio | 10%  | Organización, naming, comentarios   |

Niveles de Logro

- **Excelente (90-100%):** Supera requisitos, hooks adicionales, animaciones
- **Bueno (70-89%):** Cumple todos los requisitos base

- **Suficiente (50-69%):** Cumple requisitos mínimos con deficiencias
- **Insuficiente (<50%):** No cumple requisitos mínimos

## Tips y Mejores Prácticas

### TypeScript

```
//  Usar generics para hooks reutilizables
const useFetch = <T,>(url: string): UseFetchReturn<T> => { ... };

//  Interfaces para props
interface WidgetProps {
  title: string;
  data: DomainItem[];
  onRefresh?: () => void;
}

//  Evitar any
const data: any = response; // NO
```

### Hooks

```
//  Composición de hooks
const useWidgetData = (endpoint: string) => {
  const { data, loading } = useFetch<Item[]>(endpoint);
  const [filtered, setFiltered] = useState<Item[]>([]);
  const debouncedFilter = useDebounce(filterTerm, 300);
  // ...
};
```

### Context

```
//  Separar lógica en custom hook
const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) throw new Error('useTheme debe usarse dentro de ThemeProvider');
  return context;
};
```



## Recursos

- [React Custom Hooks](#)
- [Context API](#)
- [TypeScript Generics](#)



## Tiempo Estimado

**Total: 2-2.5 horas**

- Setup: 10 min
- Hooks: 30 min
- Context: 20 min
- Widgets: 40 min

- Integración: 20 min
- Testing/Pulido: 20 min

## **Entregables**

1. Código funcional en `starter/`
2. README actualizado con:
  - Tu dominio asignado
  - Descripción de widgets implementados
  - Screenshots del dashboard
3. Sin errores de TypeScript
4. Aplicación ejecutable con `pnpm dev`

---

**¡Éxito con tu proyecto final de Etapa 2!** 🎉